# GitHub Actions Java CI Demo Notes

**Goal:** Connect a normal Git/GitHub workflow to continuous integration (CI): push code, automatically compile/test it, see the result on a pull request, and use branch protection to prevent failed checks from being merged.

**Repository:** github-actions-demo (public)

## 1. Create the repository

1. On GitHub, create a new repository named github-actions-demo.
2. Set it to Public (or Private if you have Github pro) and check Add a README file.
3. Click Create repository.

**Why public?:** This classroom demo uses branch protection/rulesets on a public repository.

## 2. Clone the repository locally

Copy the HTTPS URL from the repository Code button, then run:

```
git clone https://github.com/YOUR-USERNAME/github-actions-demo.git
cd github-actions-demo
ls
```

**Expected:** README.md is present.

## 3. Create and run a tiny Java program

Create Calculator.java:

```
public class Calculator {
    public static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println("2 + 3 = " + add(2, 3));
    }
}
```

Compile and run it locally:

```
javac Calculator.java
java Calculator
```

**Expected output:** 2 + 3 = 5

## 4. Add a simple automated test

Create CalculatorTest.java. We use a plain Java test here so no JUnit/build-tool setup is needed.

```
public class CalculatorTest {
    public static void main(String[] args) {
        if (Calculator.add(2, 3) != 5) {
            throw new AssertionError("Test failed!");
        }

        System.out.println("All tests passed!");
    }
}
```

Compile and run the test:

```
javac Calculator.java CalculatorTest.java
java CalculatorTest
```

**Expected output:** All tests passed!

## 5. Commit and push the Java code

```
git status
git add Calculator.java CalculatorTest.java
git commit -m "Add Java calculator and test"
git push
```

**Key idea:** So far this is a normal Git workflow. The test runs only because we manually ran it.

## 6. Add a GitHub Actions workflow

Create the workflow directory:

```
mkdir -p .github/workflows
```

Create .github/workflows/java-ci.yml:

```
name: Java CI

on:
  push:
  pull_request:

jobs:
  test:
    name: Java CI Test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
```

```
      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"

      - name: Compile
        run: javac Calculator.java CalculatorTest.java

      - name: Run tests
        run: java CalculatorTest
```

### What the YAML means

| YAML | Meaning |
|---|---|
| name | Workflow name shown in the Actions tab. |
| on: push / pull_request | Run when code is pushed or a PR is opened/updated. |
| jobs | Work GitHub should perform. |
| runs-on: ubuntu-latest | Use a fresh GitHub-hosted Ubuntu runner. |
| uses | Use an existing reusable Action. |
| with | Provide configuration to an Action. |
| run | Execute a shell command on the runner. |
| Java CI Test | The job/status check that can be required before merge. |

## 7. Push the workflow

```
git status
git add .github/workflows/java-ci.yml
git commit -m "Add Java CI workflow"
git push
```

On GitHub, open the Actions tab and select the Java CI run.
Expand the steps: Checkout code → Set up Java → Compile → Run tests.

**Key idea:** The same commands we ran locally are now executed automatically on a fresh GitHub-hosted machine.

## 8. Create a feature branch

```
git checkout -b break-calculator
```

**Important:** The branch starts from the current main branch. A branch is where we can make a change without immediately changing main.

## 9. Deliberately introduce a bug

In Calculator.java, change:

```
return a + b;
```

to:

```
return a - b;
```

Run the test locally:

```
javac Calculator.java CalculatorTest.java
java CalculatorTest
```

**Expected:** The test fails with AssertionError because add(2, 3) returns -1 instead of 5.

## 10. Commit and push the broken branch

```
git status
git add Calculator.java
git commit -m "Change calculator behavior"
git push -u origin break-calculator
```

On GitHub, click Compare & pull request.
Create a PR with base: main and compare: break-calculator.

## 11. Observe CI fail on the pull request

The Java CI workflow runs automatically because the workflow includes pull_request.
Open the failed Java CI Test check and inspect the Run tests log.

**Key idea:** GitHub Actions detects and reports the failure. Without an enforcement rule, a failed check may still be mergeable.

## 12. Protect main and require the CI check

- Open the repository Settings.
- Use Branches / Branch protection rules, or Settings → Rules → Rulesets if that is what your GitHub UI shows.
- Target the main branch.
- Enable Require a pull request before merging.
- Enable Require status checks to pass before merging.
- Search for and select Java CI Test as the required status check.
- For the classroom demonstration, enable the option that prevents bypassing the rules if available, then save the rule/ruleset.

**Key distinction:** GitHub Actions reports pass/fail. Branch protection/rulesets enforce whether a PR may merge.

## 14. Confirm the broken PR is blocked

- Return to the open PR.
- The required Java CI Test check is failing, so the PR should not be normally mergeable under the protection rule.

## 15. Fix the bug and push again

On the break-calculator branch, change the method back to:

```
return a + b;
```

Run the test locally:

```
javac Calculator.java CalculatorTest.java
java CalculatorTest
```

**Expected output:** All tests passed!

Commit and push the fix:

```
git add Calculator.java
git commit -m "Fix calculator addition"
git push
```

## 16. Observe the successful PR and merge

The existing PR updates automatically when the new commit is pushed.
GitHub Actions reruns Java CI Test.
Once the check is green, the required status check is satisfied and the PR can be merged, subject to any other rules you enabled.
Merge the PR on GitHub.

**End-to-end flow:** feature branch → pull request → CI runs → failed check blocks merge → fix → CI reruns → check passes → merge.

## 17. Sync and clean up locally

```
git checkout main
git pull
git branch -d break-calculator
```

## Quick takeaways

- Run tests locally before pushing, but use CI so the repository also verifies changes automatically.
- A pull request is a review/integration point; it does not by itself guarantee correctness.
- A workflow job produces a status check. In this demo, that check is Java CI Test.
- A failed CI check reports a problem; branch protection/rulesets can make that check mandatory.
- Merge conflicts are about incompatible changes to the same code—not whether a branch changes existing code.